CSCE 479/879 Homework 1: Image Classification on the Fashion-MNIST using TensorFlow

Parker Davids, Collin Henkel, Rylen Lanning, Howraa Al-Robaie

July 25, 2025

Abstract

In this project, we designed and evaluated two fully connected neural network architectures to classify images from the Fashion-MNIST dataset [2] into 10 categories. To investigate the impact of model depth, we implemented one shallow and one deeper architecture. Each model was trained using multiple hyperparameter settings and with and without dropout and L2 regularization, resulting in eight total training runs. The best accuracy was found using the deep model, at 89.61%. However, the best shallow model performed at 89.09% accuracy, indicating that a model with higher depth may not be necessary to achieve high performance.

1 Introduction

This project uses fully connected neural networks to classify grayscale images from the Fashion-MNIST [2] dataset, containing 70,000 28×28 images of clothing items divided into 10 classes. The group trained 2 models, MLP-Shallow and MLP-Deep, with different depths and hidden layer configurations, using the TensorFlow library [1]. MLP-Shallow uses a single hidden layer, while MLP-Deep utilizes three hidden layers of decreasing node size to illustrate the effect of depth and complexity on a model's performance.

Since fully connected architectures operate using 1-dimensional inputs, both models begin by flattening the 2-dimensional images into a vector of size 784. Due to the image's relatively small size and simplicity, this proved effective. Additionally, images were normalized within the model using a rescaling layer, mapping pixel values from the range of integers [0, 255] to the range [0, 1].

Each architecture was trained under 4 conditions: the learning rates 0.001 and 0.0005, with and without regularizations of dropout = 0.5 and L2 = 1×10^{-4} , and batch sizes of 128 and 256. In total, the group ended with 8 training runs. Each run was trained on 20 epochs, with performance evaluated on a validation set and test set.

Overall, MLP-Deep had the highest accuracy, reaching 89.61% with 95% confidence that the true accuracy was between 89.01% and 90.21%. Compared with the highest MLP-Shallow performance of 89.09% accuracy, the difference

appears to be marginal, considering some shallow models even outperformed their deeper counterparts. This suggests that increased depth does not guarantee better performance for this dataset.

2 Problem Description

2.1 Learning Problem Overview

The task addressed in this project was a **supervised image classification** problem using the **Fashion-MNIST [2]** dataset. The objective was to train and evaluate different **fully connected neural network architectures** to classify grayscale images of clothing items into one of **10 predefined categories**, such as t-shirt/top, trouser, sneaker, coat, etc. Each image had a fixed dimension of 28×28 pixels, resulting in 784 input features after flattening.

This project aimed not only to maximize accuracy, but also to explore the effects of several key deep learning techniques—**dropout**, **L2 regularization**, and **learning rate adjustments**—on model generalization and training dynamics.

2.2 Dataset Description

The dataset used was the **Fashion-MNIST [2]** dataset, available through TensorFlow Datasets (TFDS) [1]. It acts as a more challenging drop-in replacement for the classic MNIST dataset. The characteristics of the dataset are as follows:

- **Number of classes:** 10 (e.g., T-shirt, Trouser, Dress, Bag, Sneaker, etc.)

Label Value	Meaning
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

- **Image size:** 28×28 pixels, grayscale
- **Training samples:** 60,000
- **Test samples:** 10,000
- **Normalization:** Pixel values scaled to $[0, 1]$, original integers from 0–255

The 60,000 training samples were split into 90% training and 10% validation using TFDS split specifications (with shuffling) to ensure a representative validation subset.

2.3 Motivation

Being more realistic and challenging than MNIST, Fashion-MNIST [2] has become widely adopted for evaluation of deep learning architectures as it provides a useful test base for examining overfitting, generalization, and the impact of various regularization techniques. Exploring these changes is essential for developing robust models that generalize ranges of data, particularly when model capacities and complexities are increased.

3 Approaches

3.1 Overall Strategy

We implemented and compared two fully connected neural network architectures, each trained with different combinations of dropout, L2 regularization, and learning rates. All models were trained using the **Adam optimizer** for 20 epochs with the `sparse_categorical_crossentropy` loss function. The evaluation was based on validation accuracy and test performance. Each model was tested at learning rates of 0.001 and 0.0005.

3.2 Model Architecture 1

Structure:

- `Flatten` input layer for 28×28 images
- `Dense(256)` with `ReLU` activation
- `Dropout(0.5)`
- `Dense(10)` with `softmax` output

Variants Tested:

- With and without `Dropout`
- With and without L2 regularization ($\lambda = 10^{-4}$)
- Learning rates: 1×10^{-3}, 5×10^{-4}; batch sizes 128, 256

Rationale: This model served as a baseline to analyze overfitting behavior in a shallow network and understand the regularization effects of dropout and L2 weight decay.

3.3 Model Architecture 2

Structure:

- `Flatten` input layer for 28×28 images
- `Dense(512)` + `ReLU` [optional Dropout]
- `Dense(256)` + `ReLU` [optional Dropout]
- `Dense(128)` + `ReLU` [optional Dropout]
- `Dense(10)` + `softmax`

Variants Tested:

- With/without dropout (0.5) and L2 ($\lambda = 0.001$)
- Learning rates: 1×10^{-3}, 5×10^{-4}; batch sizes 128, 256

Rationale: This deeper model tests whether increased capacity improves generalization on Fashion-MNIST. Dropout and L2 are used to mitigate potential overfitting given the larger number of parameters.

3.4 Justification of Design Choices

The models were built with fully connected layers and tested using different settings. We picked `ReLU` and `tanh` as activation functions because they are both commonly used and help the model learn non-linear patterns. We used the `softmax` function at the end to give class probabilities for the 10 fashion items.

We tested `dropout` with a rate of 0.5 to see if turning off random neurons during training would help the model avoid memorizing the training data. We also added `L2 regularization` with a value of $\lambda = 0.001$ to reduce overfitting by keeping the weights from getting too large.

Two learning rates, 0.001 and 0.0005, were tested to compare how fast or slow the model should update its weights. A smaller learning rate may lead to slower but more careful learning.

The different model setups helped us see which combination gave the best results without overfitting.

4 Experimental Setup

4.1 Dataset

We used the Fashion-MNIST dataset [2] consisting of 60,000 training and 10,000 test images (28×28) with 10 clothing categories (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle Boot). We carved out a 10% validation split from the training data, giving us 54,000 training examples and 6,000 validation examples.

4.2 Model Architectures

Two fully connected feed-forward architectures were created using TensorFlow and Keras [1]:

- **MLP-Shallow:** Has one hidden layer with 256 units and ReLU activation.
- **MLP-Deep:** Has three hidden layers with 512, 256, and 128 units each, all followed by ReLU.

All models concluded with a 10-class output layer. Input images were rescaled to the $[0, 1]$ range using a `Rescaling` layer at the input of each model.

4.3 Regularization

To mitigate overfitting, we experimented with:

- **Dropout:** Applied after each hidden layer with rates $\in \{0.0, 0.5\}$.
- **L2 weight decay:** Added to kernel weights with $\lambda \in \{0.0, 1 \times 10^{-4}\}$.

4.4 Hyperparameters

The following hyperparameter configurations were explored:

- **Learning rate:** 1×10^{-3}, 5×10^{-4}
- **Batch size:** 128, 256
- **Epochs:** Fixed at 20 across all runs

We used the Adam optimizer for all experiments. All training runs were capped at 20 epochs.

4.5 Evaluation Metrics

Model performance was evaluated using:

- **Test accuracy** and the corresponding 95% confidence interval, estimated via the normal approximation to the binomial distribution.
- **Confusion matrix** to identify class-specific misclassification patterns.
- **Training and validation curves** for accuracy and loss, saved as PNGs.

5 Experimental Results

5.1 Performance Comparison

Table 1 shows the test accuracy and 95% confidence intervals for all model configurations. The deep MLP without regularization achieved the highest test accuracy of **89.61%**, outperforming all shallow configurations. Interestingly, the deep model with dropout and L2 performed slightly worse, suggesting that regularization may not be necessary with sufficient data and early stopping. Among shallow MLPs, the best result came from a configuration with lower learning rate (1×10^{-3}), no regularization, and batch size 128, achieving 89.09% accuracy. All shallow models performed well, but deep models consistently outperformed them when stable.

Table 1: Test Accuracy with 95% Confidence Intervals

Model	Dropout	L2	Accuracy (%)	95% CI
MLP-Shallow (lr=1e^{-03}, bs=128)	0.00	0.0000	89.09	[88.48, 89.70]
MLP-Shallow (lr=1e^{-03}, bs=128)	0.50	0.0001	87.53	[86.88, 88.18]
MLP-Deep (lr=1e^{-03}, bs=128)	0.00	0.0000	89.57	[88.97, 90.17]
MLP-Deep (lr=1e^{-03}, bs=128)	0.50	0.0001	87.21	[86.56, 87.86]
MLP-Shallow (lr=5e^{-04}, bs=256)	0.00	0.0000	88.50	[87.87, 89.13]
MLP-Shallow (lr=5e^{-04}, bs=256)	0.50	0.0001	87.90	[87.26, 88.54]
MLP-Deep (lr=5e^{-04}, bs=256)	0.00	0.0000	89.61	[89.01, 90.21]
MLP-Deep (lr=5e^{-04}, bs=256)	0.50	0.0001	88.19	[87.56, 88.82]

95% confidence intervals computed via the normal approximation to the binomial distribution ($n = 10{,}000$).

5.2 Training Dynamics

The training and validation curves for the top-performing model (MLP-Deep without regularization) showed stable convergence, with validation accuracy closely tracking training accuracy. There was no significant overfitting observed, likely due to the relatively small model size.

Adding dropout and L2 regularization reduced test accuracy in some deep models. This suggests that the unregularized model was already generalizing well, and regularization may have limited model capacity unnecessarily, leading to mild underfitting.

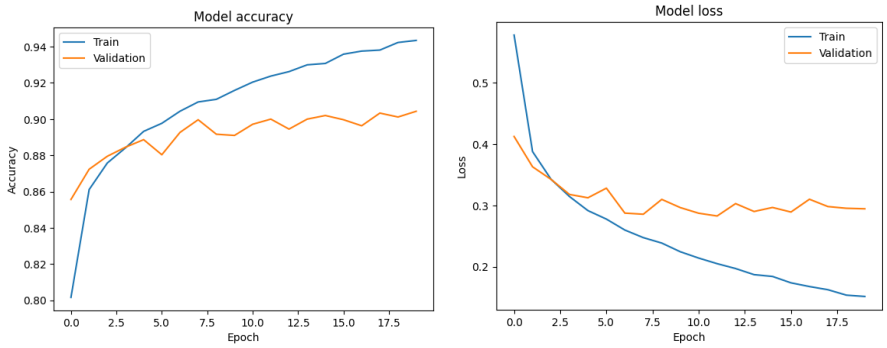

Figure 1: Training and validation accuracy/loss curves for the best-performing model (MLP-Deep, lr=5×10^{-4}, batch size=256, no regularization).

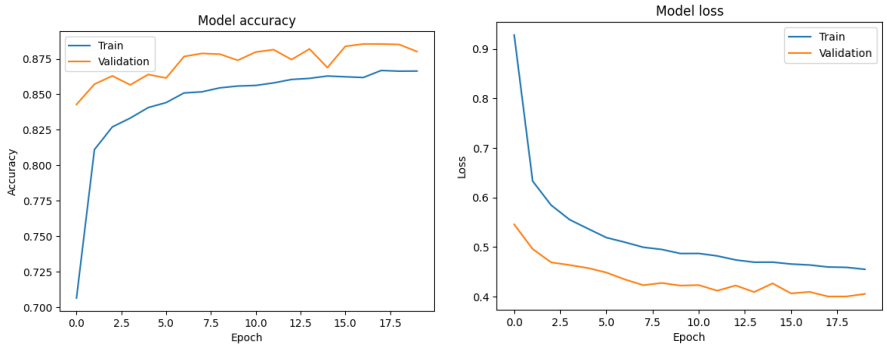

Figure 2: Training and validation curves for MLP-Deep with dropout=0.5 and L2=10^{-4}, showing slower convergence.

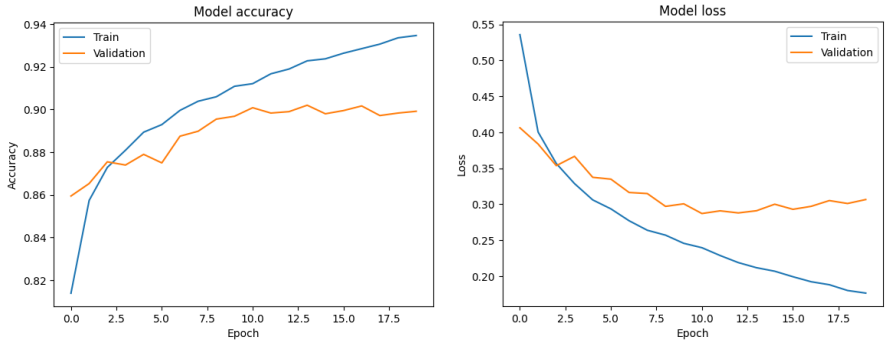

Figure 3: Training curves for MLP-Shallow (lr=10^{-3}, batch size=128, no regularization).

5.3 Error Analysis

Confusion matrix analysis revealed that most misclassifications occurred between visually similar categories, such as `Shirt` and `T-shirt/top`. Meanwhile, clearly distinct items such as `Sneaker` and `Bag` were rarely misclassified.

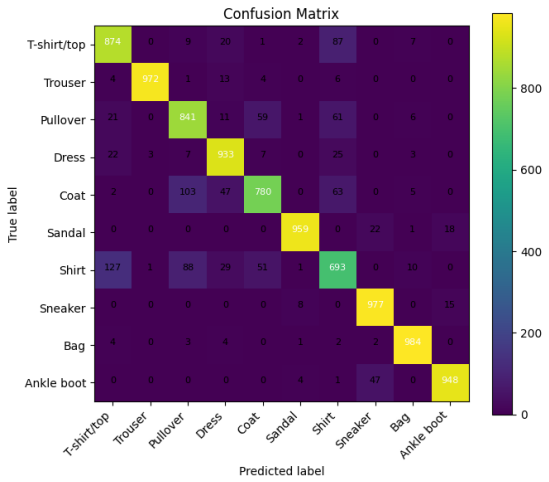

Figure 4: Confusion matrix for the best model (MLP-Deep, no regularization). Most confusion occurs between similar classes like `Shirt` and `T-shirt/top`.

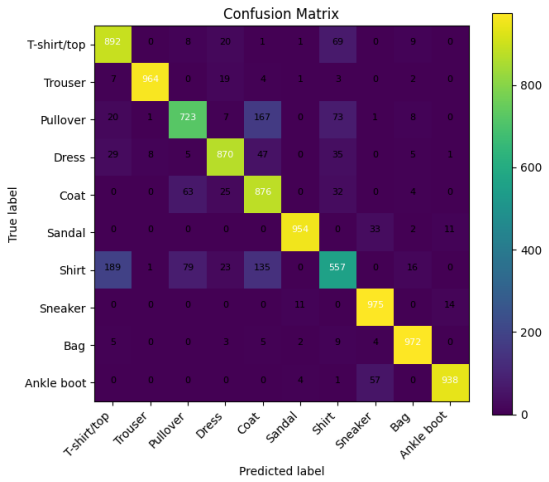

Figure 5: Confusion matrix for a regularized deep model (MLP-Deep with dropout=0.5 and L2=10^{-4}). The model makes more frequent misclassifications overall, suggesting possible underfitting.

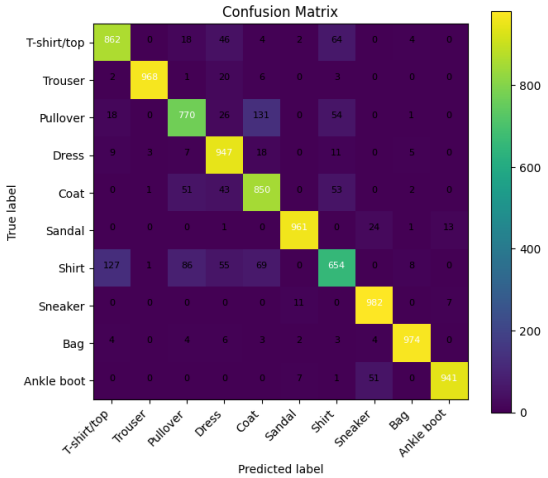

Figure 6: Confusion matrix for a shallow model (MLP-Shallow, no regularization). Errors are similar to the deep model but slightly more frequent in visually ambiguous classes.

6 Discussion

6.1 Summary

To evaluate the impact of different model architectures, hyperparameters, and regularization techniques on classification accuracy using the Fashion-MNIST [2] data set, we compared model performance across a wide variety of configurations. Our findings indicate that the MLP-Deep architecture without regularization achieved the highest test accuracy of 89.61%, out-performing the MLP-Shallow and all regularized models. Introducing dropout and L2 generally reduced test accuracy by about 1–2 percentage points, consistent with mild underfitting in this setting. Lowering the learning rate and increasing the batch size improved the deep architecture modestly; effects were mixed for the shallow model.

6.2 Key Results

Our analysis indicates that architecture and regularization techniques have a greater influence on model performance than hyperparameter settings. As shown in Table 1 introducing regularization techniques consistently led to a reduction in testing accuracy of approximately 2 percentage points. This effect was also reflected in the confidence intervals of the models, which were also shifted downwards by approximately 2 percentage points. In contrast, decreasing the learning rate while increasing the batch size, and using a deeper

architecture each resulted in an approximate 0.4 percentage point increase in test accuracy and confidence interval range.

While implementing regularization consistently led to reduced test accuracy, it also prevented the training and validation accuracies from deviating as training progressed. This indicates that regularization effectively prevented overfitting at the cost of model performance.

6.3 Limitations

This study has several limitations that should be considered. First, all models were trained for only twenty epochs, which may not have been sufficient for overfitting to occur, particularly in models that did not use regularization. Second, both dropout and L2 regularization were applied simultaneously when regularization was enabled. This prevented us from exploring the individual contribution of either regularization technique on model performance.

6.4 Future Work

Future work can address several limitations of this study. First, training models for a greater number of epochs may provide insights into the long-term performance of the deep and shallow architectures, as well as gain a clearer picture of how overfitting might emerge. This could help in understanding the long-term benefits of applying dropout and L2 regularization. Second, to better assess the individual impact of dropout and L2 regularization, future studies should apply each technique separately and together. This would illuminate the specific impact each regularization technique has on the generalized performance.

7 Conclusions

This project evaluated the performance of two fully connected neural network architectures on the Fashion-MNIST [2] dataset. Each architecture was trained using two learning rates, two batch sizes, and with or without regularization. The highest accuracy was achieved by our deep model, reaching 89.61%. However, the shallow model's best score came at a close second of 89.09%, and in some instances even performed better than its corresponding deep model. This suggests that a deeper architecture may not always lead to better performance and may be more sensitive to its hyperparameters. Also, regularization through L2 and dropout appeared to reduce the accuracy across both models in this dataset. If we were to continue this project, we would try implementing other regularization techniques and activation functions such as early stopping and leaky ReLU. A convolutional architecture may have also proved beneficial.

Table 2: Contributions by team member for this assignment.

Team Member	**Contribution**
Howraa Al-Robaie	Wrote the Problem Description and Approaches sections of the report. Took the lead on building, training, and debugging both neural network architectures, running multiple experiments, and generating evaluation plots.
Rylen Lanning	Cleaned up the codebase and incorporated L2 regularization into the models. Fixed Confusion Matrix Plots. Handled submission preparation and contributed to the Experimental Setup and Results sections.
Parker Davids	Assisted with error checking and verification of model training scripts. Contributed to the Discussion and Conclusions sections of the report.
Collin Henkel	Helped initialize the project setup, reviewed and debugged model code, and contributed to the Abstract, Introduction, Conclusion, and Appendix sections of the report.

A Appendix

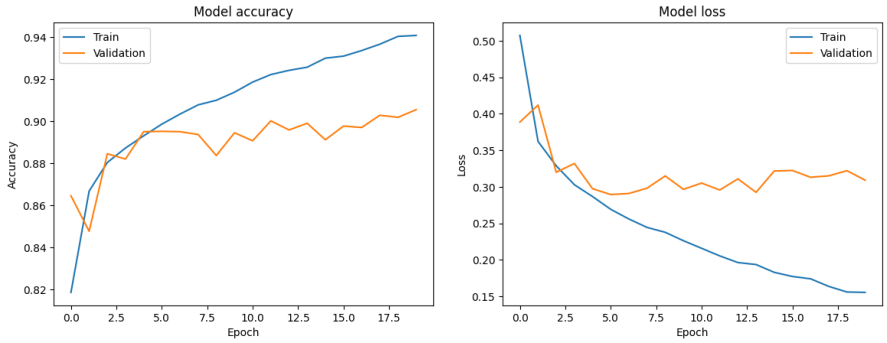

Figure 7: Training and validation accuracy/loss for MLP-Deep (lr=1×10^{-3}, batch size=128, no regularization).

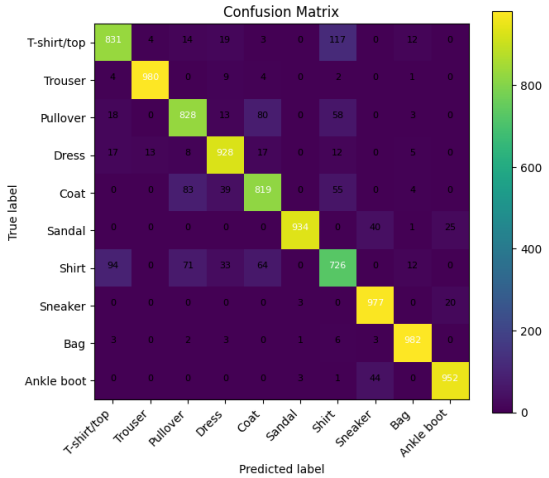

Figure 8: Confusion matrix for MLP-Deep (lr=1×10^{-3}, batch size=128, no regularization)

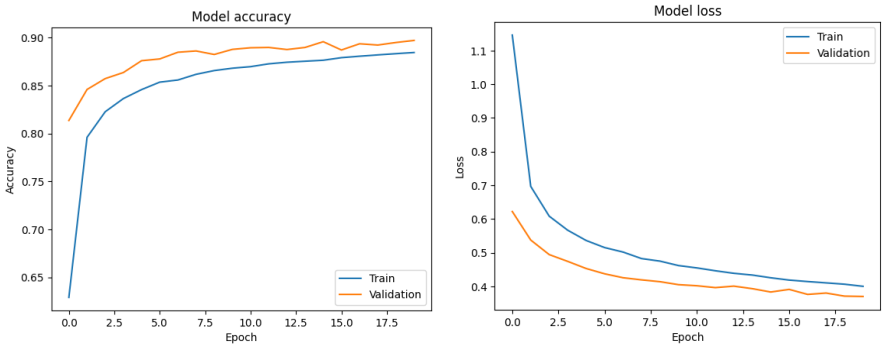

Figure 9: Training and validation accuracy/loss curves for (MLP-Deep, lr=5×10^{-4}, batch size=256, dropout=0.5, and L2=10^{-4}).

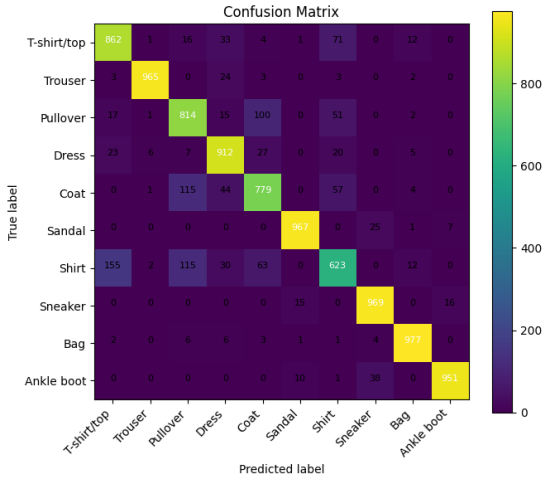

Figure 10: Confusion matrix for (MLP-Deep, lr=5×10^{-4}, batch size=256, dropout=0.5, and L2=10^{-4}).

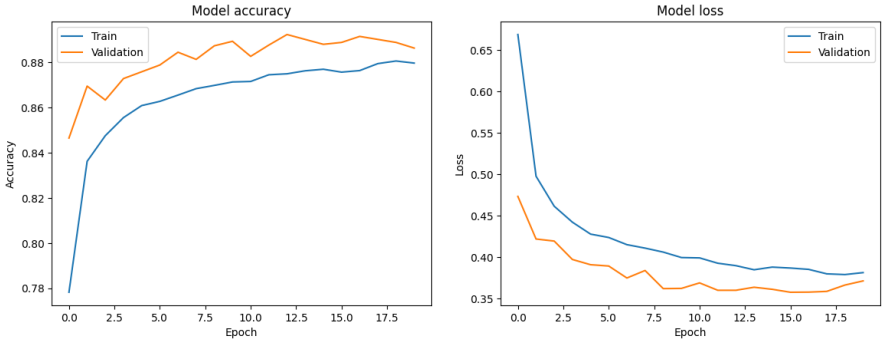

Figure 11: Training and validation accuracy/loss curves for (MLP-Shallow, lr=1×10^{-3}, batch size=128, dropout=0.5, and L2=10^{-4}).

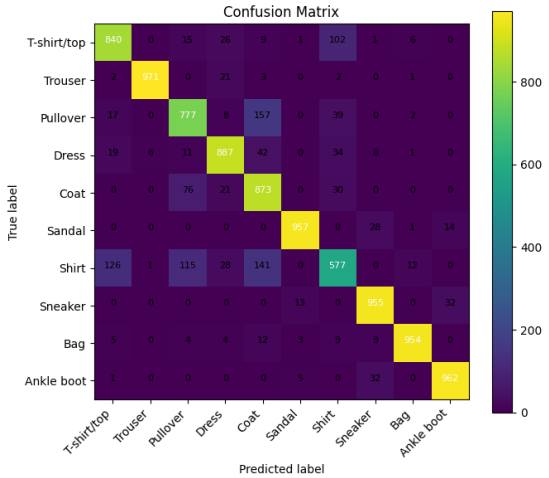

Figure 12: Confusion matrix for (MLP-Shallow, lr=1×10^{-3}, batch size=128, dropout=0.5, and L2=10^{-4}).

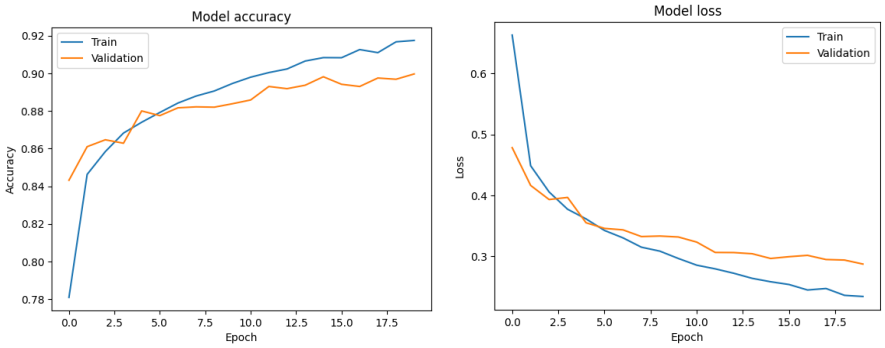

Figure 13: Training and validation accuracy/loss curves for (MLP-Shallow, lr=5×10^{-4}, batch size=256, no regularization).

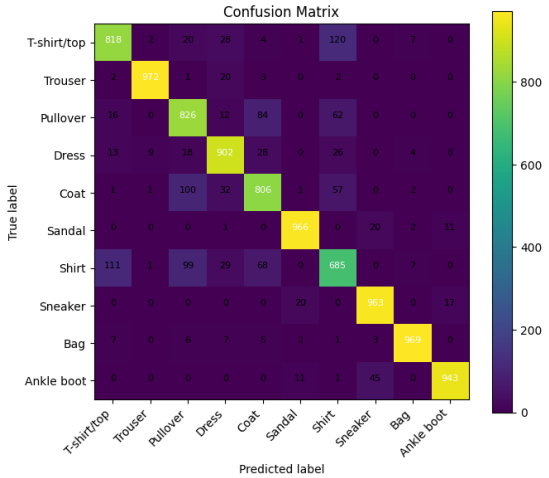

Figure 14: Confusion matrix for (MLP-Shallow, lr=5×10^{-4}, batch size=256, no regularization).

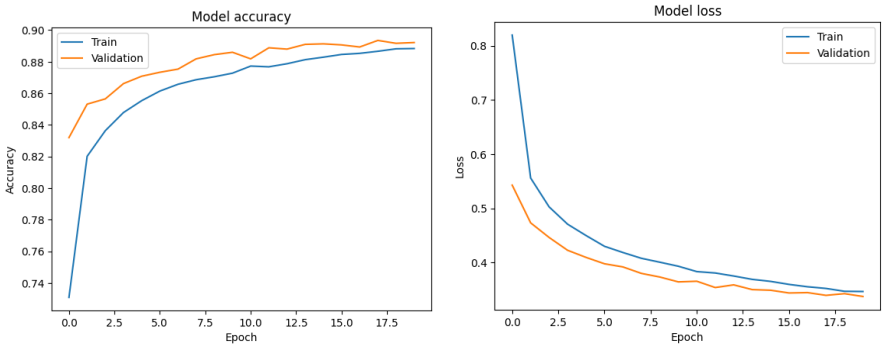

Figure 15: Training and validation accuracy/loss curves for (MLP-Shallow, lr=5×10^{-4}, batch size=256, dropout=0.5, and L2=10^{-4}).

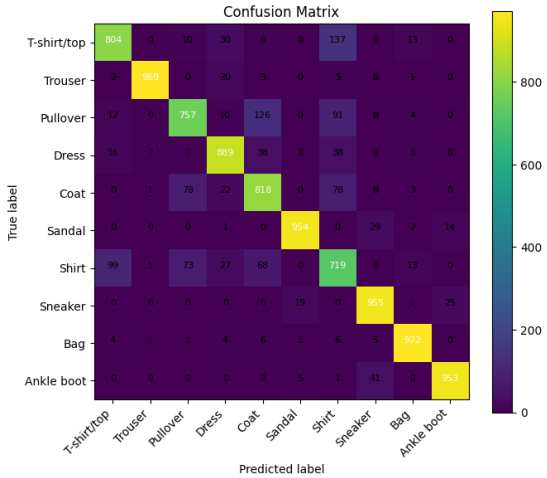

Figure 16: Confusion matrix for (MLP-Shallow, lr=5×10^{-4}, batch size=256, dropout=0.5, and L2=10^{-4}).

References

[1] TensorFlow/Keras, "TensorFlow," 2024. [Online]. Available: `https://www.tensorflow.org/` (accessed Jul. 24, 2025).

[2] Zalando Research, "Fashion-MNIST dataset," 2017. [Online]. Available: `https://github.com/zalandoresearch/fashion-mnist` (accessed Jul. 24, 2025).

[3] TensorFlow, "TFDS: Fashion-MNIST documentation," 2024. [Online]. Available: `https://www.tensorflow.org/datasets/catalog/fashion_mnist` (accessed Jul. 24, 2025).

[4] Scikit-learn Developers, "Scikit-learn: Machine learning in Python," 2024. [Online]. Available: `https://scikit-learn.org/` (accessed Jul. 24, 2025).

[5] J. D. Hunter *et al.*, "Matplotlib: Visualization with Python," 2024. [Online]. Available: `https://matplotlib.org/` (accessed Jul. 24, 2025).